The Islamia University of Bahawalpur
Department of Information Technology

INFT-4041 · BSIT · 5th Semester · Fall 2026

E-Commerce Application Development

Chapter

15

User Account Management

Part IV — Money and Identity

Taught in	Week 11
Laboratory	Password cost against logins per second
Assessment	—

This is a class extract. It is *Chapter 15* of the unified handout, printed on its own so that a week's reading is a week's reading. Page numbers restart at 1 and are local to this extract; chapter, figure and table numbers are the ones the full volume gives them, so a reference you follow here lands in the same place there.

Cross-references to other chapters print as their numbers in the full volume. For the whole course in one file — front matter, the HEC tracing table, all twenty-three chapters and all seven appendices — see `ECAD_Fall2026_Handout.pdf`.

Dr. Mustafa Hameed · Department of Information Technology

Generated from `parts/` by `makechapters.py`; do not edit this PDF's source by hand

Chapter 15

User Account Management

Part 1 | Part 2 | Part 3 | **Part 4** | Part 5 | Part 6

Part IV — Money and Identity

Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why a password is stored as a deliberately slow hash, and **name** what the cost factor buys.
- **Measure** the cost factor against throughput, and **choose** one for a stated shop.
- **Demonstrate** user enumeration by timing, and **explain** why a stronger hash makes it worse.
- **Evaluate** a lockout policy, and **identify** what it enables as well as what it prevents.
- **Design** a password reset flow that does not become the weakest way in.

Before You Start

Chapter 9: sessions, cookies, and that the server decides what is true. Chapter 14: measuring a distribution rather than a single value. Appendix A — every exercise here runs against your own container.

Key Terms in This Chapter

authentication • password hash • bcrypt • cost factor • salt • rainbow table • credential stuffing • user enumeration • timing side channel • lockout • rate limiting • password reset • account takeover

15.1 Why a Password Is Stored Slowly on Purpose

Every other piece of engineering in this course tries to make something faster. Password storage is the one place where the software is deliberately slowed down, and understanding why is the whole of the chapter.

Hash, Salt, Cost Factor

A **password hash** is a one-way function of the password. The shop stores the output and never the password, so that a stolen database does not hand the attacker the passwords.

A **salt** is a random value mixed in, different for every user, so that two customers with the same password get different hashes — and so that an attacker cannot precompute a table of hashes once and look everybody up.

A **cost factor** says how much work the function does. Bcrypt's is a power of two: raising it by one **doubles** the time. That is a cost to your server once per login and a cost to the attacker once per *guess*, and the attacker is making billions of them.

What this shop actually does

- Passwords are stored with PHP's `password_hash(..., PASSWORD_DEFAULT)`, which here produces a `$2y$10$` hash — **bcrypt at cost 10**, PHP's default and unchanged since 2013.

- The column is `varchar(255)`, which is correct: it leaves room for whatever algorithm PHP's default becomes next.
- Version 3 of this platform used SHA-1 with a salt, which is far too fast to be a password hash at all. Version 4 moved to `password_hash`. **When you inherit a shop, check which one it is before anything else.**
- The minimum password length is **6 characters**, and none of the complexity settings — uppercase, lowercase, number, symbol — is set at all, so none is enforced.

Measured: The cost factor against the clock

From `code/m15_accounts.py -cost`, timed by PHP inside the shop container on the machine in Appendix A. "Logins per second" is one core doing nothing else.

Cost	Hash	Verify	Logins/sec	Work vs cost 8
8	13.6 ms	13.7 ms	73	1×
9	29.6 ms	26.8 ms	37	2×
10	**52.8 ms**	**60.3 ms**	**17**	4×
11	110.2 ms	103.0 ms	10	8×
12	217.5 ms	218.9 ms	5	16×
13	440.7 ms	448.2 ms	2	33×
14	846.8 ms	873.8 ms	1	64×

The doubling is exact, which is what a well-behaved cost factor looks like: 13.7, 26.8, 60.3, 103, 219, 448, 874. Nothing else in this course produces a table that clean, because nothing else in this course is a cost that somebody designed on purpose.

The shop ships at cost 10: 60 ms per login, 17 logins per second on one core. A shop with 17 simultaneous login attempts is a shop in trouble for other reasons, so cost 10 is not a throughput problem — but it is also twelve-year-old advice, and hardware has not stood still.

Cost 12 is the usual current recommendation: 219 ms, 5 logins per second. For a shop taking a few hundred orders a day that is free, and it makes every stolen hash sixteen times more expensive to attack than cost 8. **Raising it is one line, and the next section is the reason to think first.**

15.2 What the Login Form Tells an Attacker Who Has No Password

An attacker with a list of email addresses and no passwords still wants something from your login form: *which of these addresses have accounts here.* That list is worth money on its own, and it makes every later attack cheaper.

Measured: A registered email answers more slowly, and measurably so

From `code/m15_accounts.py -enumerate`: fifteen login attempts with a deliberately wrong password, against an address that exists and one that does not, timed from outside the container as an attacker would.

	Median	Fastest	Slowest
Registered email	**88.6 ms**	70.9 ms	142.7 ms
Unknown email	**23.0 ms**	15.2 ms	94.8 ms

A gap of 65.5 ms — 74% of the slower time — visible from outside, over the network, without a single correct password.

Where the 65 ms comes from is the uncomfortable part. It is almost exactly the cost of one bcrypt verify at cost 10, measured in the previous section as 60.3 ms. A wrong password on a real account must be checked against the stored hash; a wrong *email* matches no row, so there is nothing to check and the request returns early.

So the leak is produced by the security measure. Raise the cost factor to 12 for the excellent reasons above, and the gap grows from 65 ms to roughly 220 ms — four times louder, trivially measurable over any network. A change that makes stolen hashes sixteen times harder to crack makes your customer list four times easier to enumerate.

The fix is one line and it is not obvious: when the email matches no customer, verify the supplied password against a *dummy hash* anyway and discard the result. The request then costs the same whether the account exists or not. It feels like wasted work because it is wasted work — that is the point.

Timing is not the only way this leaks

Closing the timing channel is necessary and not sufficient. The same information escapes through:

- **Different messages.** "No such account" versus "Wrong password" states it outright. Use one message for both.
- **Registration.** "That email is already registered" is the same disclosure through a different form, and it is harder to remove, because the user genuinely needs to know. Send an email instead of answering on the page.
- **Password reset.** "We have sent you a link" must be shown whether or not the address exists.
- **The lockout itself.** See below — a locked account answers differently from one that was never there.

Enumeration is rarely the attack. It is the reconnaissance that makes the attack affordable, and that is why it is worth closing even though no single leak looks serious.

15.3 Lockout: What It Prevents, and What It Enables

Measured: Five wrong guesses, then what?

From `code/m15_accounts.py -lockout`. The shop's `config_login_attempts` is **5**.

Attempt	Time	Recorded	Result
1–5	70–81 ms	1, 2, 3, 4, 5	rejected, hash verified each time
6	**16 ms**	5	**locked**
7	**15 ms**	5	locked
correct password	**17 ms**	5	**still refused**

It works. After five failures the account stops answering, for an hour, keyed on the pair (email, IP address).

Two things the table shows that the setting does not.

First, the correct password is refused too. That is right — a lock that the attacker's next guess could open would be no lock — but it means **anyone who knows a customer's email address can lock them out of their own account at will**, with five wrong guesses, for an hour, repeatedly. Lockout converts a password-guessing attack into a denial-of-service one. It is the right trade for most shops and it is a trade, not a free win.

Second, the locked response is four times faster, because it never reaches the hash. The lockout is itself an enumeration channel: 16 ms says "this account exists and is locked", which is more than the 23 ms of an unknown address was giving away.

And the key is (email, IP). An attacker with a few hundred addresses has effectively no limit, while a school or an office behind one NAT shares a single counter — so the mechanism is weakest against the attacker it is for and strongest against the customers it is not.

Where the real risk is, and it is not your password form

Customers reuse passwords. An attacker who has a list of email and password pairs stolen from somewhere else tries them on your shop — **credential stuffing** — and some of them work, because the passwords are genuinely correct. Nothing in this chapter prevents that: the hash is fine, the lockout counts five failures and these are not failures.

What helps is **not** another password rule. It is: checking new passwords against lists of known-breached ones; offering a second factor; rate limiting by IP and by address across the whole site rather than per account; and noticing the pattern — many accounts, one address, high success rate — which is a Chapter 17 problem rather than a login problem.

Meanwhile this shop's minimum password length is six characters with no complexity requirement at all. That is the setting to change first, and it is in **System › Settings › Edit › Option**.

The reset flow is the other front door

An account is only as secure as the cheapest way to get into it, and that is usually the password reset.

1. **The token must be unguessable and single-use**, and must expire in minutes rather than days.
2. **It must not be in the URL of a page that loads third-party scripts**, which leak it in the `Referer` header.
3. **Resetting must invalidate existing sessions**, or an attacker who was already in stays in after the customer "fixes" it.
4. **The response must not reveal whether the address exists**, which is the enumeration problem again, in its easiest-to-fix form.
5. **The email itself is now the weakest link.** You have delegated your authentication to the customer's mail provider, and should know that you have done so.

Common Pitfalls

- **Choosing a cost factor without measuring it on the machine that will run it.** Cost 14 is one login per second.
- **Raising the cost factor without closing the timing channel**, which makes enumeration four times easier while you congratulate yourself.

- **Different error messages for "no account" and "wrong password".**
- **Believing lockout is free.** It hands anyone a denial-of-service tool against any customer whose email they know.
- **Rate limiting only per account**, which does nothing against an attacker trying one password across ten thousand accounts.
- **Leaving the minimum password length at six with no complexity rules**, which is this shop's shipped state.
- **A reset token that does not expire**, or that survives being used.
- **Testing any of this against a shop that is not yours.** See Appendix A.

Try It Yourself: Measure your own login, then make it quieter

```
# Everything here runs against YOUR container on localhost. Read the
# scope statement in Appendix A before you start.

# --- 1. Find out what your shop stores ------------------------------
cd code
docker compose exec db mariadb -ushop -pshoppw opencart \
  -e "SELECT email, LEFT(password,7) FROM oc_customer LIMIT 3;"
# The first seven characters name the algorithm and the cost.

# --- 2. Measure the cost factor -------------------------------------
python m15_accounts.py --cost

# --- 3. Choose one, and justify it in a sentence --------------------
# How many logins per second does your shop actually need? Work it
# out from orders per day, not from a feeling.

# --- 4. Predict, then measure the timing leak -----------------------
# Does a wrong password on a REAL account take longer than one on an
# address that does not exist? Predict yes or no, and by how much.
python m15_accounts.py --enumerate

# --- 5. Explain the number you got ----------------------------------
# Compare the gap with the verify time from step 2. They should
# match. Say why.

# --- 6. Watch the lockout, and find what it gives away --------------
python m15_accounts.py --lockout
# Two separate problems are visible in that table. Name both.

# --- 7. Tighten the policy ------------------------------------------
# Admin: System > Settings > Edit > Option. Raise the minimum
# password length and turn on the complexity requirements. Then
# register a new customer and confirm the rules are enforced.

# --- 8. Write the fix you cannot make in the admin panel ------------
# Three lines of pseudocode: how would you make a login to a
# non-existent account take the same time as one to a real account?

# --- 9. Clean up ----------------------------------------------------
python m15_accounts.py --clean
```

What to notice. Step 5. The timing leak is not a bug somebody left in — it is the direct, unavoidable consequence of doing the right thing with passwords. Security measures interact, and sometimes they interact badly.

Checkpoint — Test Yourself

1. Your shop runs bcrypt at cost 10 and takes 400 orders a day. Work out whether cost 12 is affordable, stating your assumptions about logins per order and peak concentration, and say what else you would have to change at the same time and why.
2. A wrong password on a registered address took 88.6 ms and on an unknown one 23.0 ms. Explain the cause, state what an attacker does with that, and give the one-line fix together with the reason it feels wrong to write.
3. A client asks you to remove the five-attempt lockout because customers complain about being locked out. Give the risk you are accepting, the risk you are removing, and a design that addresses both complaints better than either keeping or removing it.

Chapter Summary

- A password hash is slow on purpose. The salt defeats precomputation; the cost factor sets how expensive each guess is, and each step doubles it for you and for the attacker.
- This shop uses `password_hash` with bcrypt at cost 10, in a `varchar(255)` column. Version 3 used salted SHA-1; check which one any shop you inherit is using before anything else.
- Measured: cost 8 to 14 ran 13.7, 26.8, 60.3, 103, 219, 448 and 874 ms — an exact doubling. Cost 10 gives 17 logins per second on one core; cost 12, the usual current recommendation, gives 5.
- Measured: a wrong password on a registered address took 88.6 ms and on an unknown address 23.0 ms — a 65 ms gap, visible over the network, and almost exactly one bcrypt verify.
- The leak is caused by the defence. Raising the cost factor to 12 would make the gap roughly 220 ms. The fix is to verify against a dummy hash when no account matches, so that both paths cost the same.
- Enumeration also leaks through error messages, registration, password reset and the lockout response. It is reconnaissance, not the attack.
- Measured: five wrong guesses lock the account for an hour, keyed on (email, IP). The correct password is refused too — so lockout hands an attacker a denial-of-service tool — and the locked reply is four times faster, which is itself a disclosure.
- Credential stuffing defeats all of it, because the passwords are correct. The answers are breach-list checks, a second factor, site-wide rate limiting, and noticing the pattern.
- The password reset is the other front door: single-use, short-lived tokens; session invalidation; and the knowledge that you have delegated authentication to a mail provider.

Review Questions

1. (MCQ) Raising bcrypt's cost factor from 10 to 12 multiplies the work by: (a) 1.2; (b) 2; (c) 4; (d) 16.

2. (MCQ) The measured 65 ms gap between a registered and an unknown email is closest to: (a) network latency; (b) one bcrypt verify; (c) the database lookup; (d) template rendering.

3. (Short) Explain what a salt prevents, and why it does not need to be secret.

4. (Short) Give two ways a shop leaks which email addresses are registered, other than by timing, and say how to close each.

5. (Short) Explain why account lockout is a trade rather than an improvement, naming what it enables.

6. **(Applied)** A shop's database is stolen. All passwords are bcrypt at cost 10. Explain what the attacker can and cannot do, what the shop must tell its customers, and what it should change — and say why changing the cost factor now helps less than you might expect.

7. **(Applied)** Design rate limiting for a login form that must stop credential stuffing without locking out legitimate customers. State what you count, over what window, keyed on what, what happens at the limit, and how a school computer lab with forty students behind one address is not broken by your design.